

# Зачем мы вообще говорим о неструктурированных данных?

- У компаний тонны документов, но мало «живого» доступа к знаниям.
- Поиск «на глаз» и обычный full-text плохо масштабируются.
- LLM без доступа к вашим документам галлюцинируют.
- RAG превращает хаос документов в рабочего ассистента.

# Структурированные и неструктурированные данные

- Структурированные: таблицы и БД (строгое поле → значение).
- Полу-структурированные: JSON, XML, лог-файлы.
- Неструктурированные: текст, PDF, Word, PPT, сканы, аудио/видео.
- RAG чаще всего нужен именно для неструктурированных данных.

# Базовая идея RAG: Retrieval + Generation

- LLM сильна в генерации, но не знает ваши внутренние документы.
- Retrieval достаёт релевантные куски из базы знаний.
- LLM формирует ответ, опираясь на эти куски.
- Результат: меньше галлюцинаций, больше точности.

# RAG-пайплайн для документов

- 1) Собрать документы.
- 2) Извлечь текст и структуру.
- 3) Очистить и нормализовать текст.
- 4) Разбить на чанки и добавить метаданные.
- 5) Построить эмбеддинги и векторный индекс.
- 6) При запросе: найти релевантные чанки.
- 7) Передать чанки и вопрос в LLM и получить ответ.

# Где в этой схеме появляется LangChain

- LangChain — каркас вокруг LLM и данных.
- Компоненты: loaders, text splitters, embeddings, vector stores, chains.
- Упрощает работу с разными форматами и провайдерами.

# Форматы, с которыми работаем в модуле

- Excel — отзывы, выгрузки, отчёты.
- Word — контракты, регламенты, инструкции.
- PowerPoint — презентации, стратегии, отчёты.
- EPUB — книги, учебные материалы.
- PDF — текстовые и сканированные документы.